

HANDS-ON PROJECT • BUILD & LEARN

College Management System

Database Project

Learn every database concept by building a real system from scratch

ACID

Rows

Columns

Primary Keys

Foreign Keys

Schema

Indexes

Constraints

Relationships

Step-by-Step • Full SQL Code • Sample Data • Practice Tasks • Exercises

Section 1 — Core Concepts

What You Will Build

You will build a **College Management System** database from scratch. By the end of this project you will have designed, created, populated, queried, and secured a complete relational database — learning every core concept naturally as you need it.

Phase	Step	What You Build	Concept Learned
1 — Design	Step 1	Identify entities & draw the schema	Tables, Schema
2 — Create	Step 2	Create the database and all tables	Columns, Data Types
2 — Create	Step 3	Add Primary Keys to every table	Primary Key
2 — Create	Step 4	Link tables with Foreign Keys	Foreign Key, Relationships
2 — Create	Step 5	Add all data-quality constraints	Constraints
3 — Populate	Step 6	Insert realistic sample data	Rows / INSERT
3 — Populate	Step 7	Update and delete rows	UPDATE, DELETE
4 — Optimise	Step 8	Create indexes for fast queries	Index
5 — Query	Step 9	Write queries across all tables	SELECT, JOINS
6 — Transact	Step 10	Use transactions for safe updates	ACID Properties
7 — Practice	Exercises	20 practice tasks to reinforce learning	All Concepts

Schema Design

PHASE 1

Identify real-world entities and plan the database structure before writing any SQL

Concepts covered: [Tables](#) • [Schema](#) • [Relationships \(1:1, 1:N, M:N\)](#)

Step 1 — Identify Entities and Relationships

Before writing a single line of SQL, identify the real-world **things** your system needs to track. Each thing becomes a **table**. Connections between them become **relationships**.

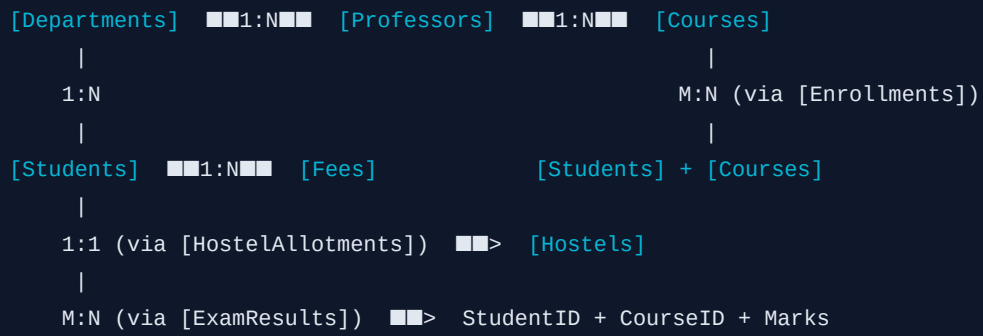
Entity (Table)	What It Stores	Real-World Meaning
Departments	Academic departments in the college	Computer Science, Maths, Physics...
Professors	Teaching staff	Name, email, which dept they belong to
Students	Enrolled students	Name, roll number, CGPA, dept
Courses	Subjects offered	Title, credits, taught by which professor
Enrollments	Which student is in which course	Bridge table for M:N
Hostels	On-campus student housing	Hostel name, capacity, warden
HostelAllotments	Which student lives in which hostel	1:1 bridge
ExamResults	Marks per student per course	Internal, external, total
Fees	Fee payment records	Amount, date, status per student

Relationship Map

Table A	Relationship	Table B	Type	How Implemented
Departments	has many	Professors	1 : N	DeptID FK in Professors
Departments	has many	Students	1 : N	DeptID FK in Students
Professors	teaches many	Courses	1 : N	ProfID FK in Courses
Students	enrols in many	Courses	M : N	Enrollments junction table
Students	lives in one	Hostels	1 : 1	HostelAllotments table (StudentID UNIQUE)
Students	pays many	Fees	1 : N	StudentID FK in Fees
Students & Courses	has result	ExamResults	M : N extended	StudentID+CourseID FK pair

Schema Diagram

SCHEMA DIAGRAM



PHASE 2

Concepts covered: Columns • Data Types • Primary Key • Foreign Key • Constraints • Schema

PHASE 3

Concepts covered: Rows • INSERT • UPDATE • DELETE

[illegible][illegible][illegible]

Step 8 — INSERT More Data: Results, Hostels, Fees

Step 9 — UPDATE and DELETE Operations

Page 7

PHASE 4

Concepts covered: Index • Query Performance • EXPLAIN

Think about which columns you search on most often — those are your index candidates.

[illegible]

Index Created On	Speeds Up This Query	Without Index	With Index
Students(RollNo)	SELECT * WHERE RollNo='CSE20210912'	Scans all students	Instant lookup
Students(DeptID)	SELECT * WHERE DeptID = 1	Scans all students	Jumps to dept group
ExamResults(StudentID,Semester)	Results for one student this sem	Full scan	Direct range scan
Fees(StudentID,Status)	All pending fees for a student	Full scan	Filtered index scan
Courses(Title) FULLTEXT	MATCH(Title) AGAINST('machine')	LIKE '%machine%' slow	Fast word lookup

Query the Database

PHASE 5

Write SELECT queries — simple, JOIN, GROUP BY, subquery

Concepts covered: **SELECT • JOIN • WHERE • GROUP BY • Subqueries**

Step 11 — Basic Queries

```
-- Q1: List all active students with their department SELECT s.RollNo, s.FullName,
d.DeptName, s.Batch, s.CGPA FROM Students s JOIN Departments d ON d.DeptID = s.DeptID WHERE
s.IsActive = TRUE ORDER BY s.CGPA DESC; -- Q2: All courses taught by a specific professor
SELECT p.FullName AS Professor, c.CourseCode, c.Title, c.Credits FROM Professors p JOIN
Courses c ON c.ProfID = p.ProfID WHERE p.ProfID = 1; -- Q3: All students enrolled in DBMS
(CS302) SELECT s.RollNo, s.FullName, e.Semester, e.Status FROM Students s JOIN Enrollments
e ON e.StudentID = s.StudentID JOIN Courses c ON c.CourseID = e.CourseID WHERE c.CourseCode
= 'CS302' ORDER BY s.FullName;
```

Step 12 — Aggregate and Group Queries

```
-- Q4: Count of students per department SELECT d.DeptName, COUNT(s.StudentID) AS
TotalStudents, ROUND(AVG(s.CGPA),2) AS AvgCGPA FROM Departments d LEFT JOIN Students s ON
s.DeptID = d.DeptID AND s.IsActive = TRUE GROUP BY d.DeptID, d.DeptName ORDER BY
TotalStudents DESC; -- Q5: Top 3 students by CGPA in CSE department SELECT s.RollNo,
s.FullName, s.CGPA FROM Students s JOIN Departments d ON d.DeptID = s.DeptID WHERE
d.DeptCode = 'CSE' AND s.IsActive = TRUE ORDER BY s.CGPA DESC LIMIT 3; -- Q6: Total fees
collected vs pending per department SELECT d.DeptName, SUM(CASE WHEN f.Status='Paid' THEN
f.Amount ELSE 0 END) AS Collected, SUM(CASE WHEN f.Status='Pending' THEN f.Amount ELSE 0
END) AS Pending FROM Departments d JOIN Students s ON s.DeptID = d.DeptID JOIN Fees f ON
f.StudentID = s.StudentID GROUP BY d.DeptName;
```

Step 13 — Advanced Queries (Subqueries, Multi-join)

```
-- Q7: Students with exam total > 80 in CS301 (Data Structures) SELECT s.FullName,
r.Internal, r.External, r.Total, r.Grade FROM ExamResults r JOIN Students s ON s.StudentID =
r.StudentID JOIN Courses c ON c.CourseID = r.CourseID WHERE c.CourseCode = 'CS301' AND
r.Total > 80 ORDER BY r.Total DESC; -- Q8: Students who have NOT paid any fees (subquery)
SELECT s.RollNo, s.FullName, d.DeptName FROM Students s JOIN Departments d ON d.DeptID =
s.DeptID WHERE s.StudentID NOT IN ( SELECT DISTINCT StudentID FROM Fees WHERE Status =
'Paid' ); -- Q9: Full student profile – name, dept, hostel, CGPA, fees due SELECT s.RollNo,
s.FullName, d.DeptName, h.HostelName, ha.RoomNo, s.CGPA, COUNT(CASE WHEN f.Status='Pending'
THEN 1 END) AS PendingFees FROM Students s LEFT JOIN Departments d ON d.DeptID = s.DeptID
LEFT JOIN HostelAllotments ha ON ha.StudentID = s.StudentID LEFT JOIN Hostels h ON
h.HostelID = ha.HostelID LEFT JOIN Fees f ON f.StudentID = s.StudentID GROUP BY s.StudentID,
s.RollNo, s.FullName, d.DeptName, h.HostelName, ha.RoomNo, s.CGPA ORDER BY s.CGPA DESC;
```

Transactions & ACID

PHASE 6

Use transactions to make multi-step operations safe and reliable

Concepts covered: ACID • BEGIN • COMMIT • ROLLBACK • SAVEPOINT

Step 14 — Transaction 1: Student Enrolment (Atomicity)

When a new student registers, several tables must be updated together. If any step fails, **all steps are undone**.

```
START TRANSACTION; -- all or nothing -- Step A: Add the new student INSERT INTO Students
(RollNo, FullName, Email, Phone, DateOfBirth, Gender, DeptID, Batch) VALUES
('CSE2024001', 'Vikram Singh', 'vikram.s@student.edu', '8900001200', '2006-04-10', 'M', 1, 2024);
-- Step B: Enrol in mandatory first-year courses INSERT INTO Enrollments (StudentID,
CourseID, Semester) VALUES (LAST_INSERT_ID(), 1, '2024-Odd'), (LAST_INSERT_ID(), 3,
'2024-Odd'); -- Step C: Create tuition fee record INSERT INTO Fees (StudentID, Amount,
FeeType) VALUES (LAST_INSERT_ID(), 85000, 'Tuition'); COMMIT; -- all 3 steps saved together
-- If step B fails (e.g. course full), ROLLBACK ensures -- student is NOT added without
courses and fee record
```

Step 15 — Transaction 2: Fee Payment (Consistency + Durability)

```
START TRANSACTION; -- Verify student exists and fee is truly pending SELECT Status FROM Fees
WHERE FeeID = 5 AND Status = 'Pending'; -- Application checks: if 0 rows returned →
ROLLBACK (already paid) -- Mark fee as paid UPDATE Fees SET Status = 'Paid', PaidOn =
CURRENT_DATE WHERE FeeID = 5; -- Log the payment in a receipts table INSERT INTO Receipts
(FeeID, StudentID, Amount, PaymentMode, TxnRef) VALUES (5, 4, 70000, 'Online',
'TXN20240612ABC'); COMMIT; -- Payment record is durable – survives any server crash
```

Step 16 — Transaction 3: Exam Result Entry (SAVEPOINT)

```
START TRANSACTION; -- Enter internal marks for a batch of students INSERT INTO ExamResults
(StudentID, CourseID, Semester, Internal) VALUES (5, 2, '2024-Odd', 30); SAVEPOINT
internal_done; -- safe checkpoint after internal marks INSERT INTO ExamResults (StudentID,
CourseID, Semester, Internal) VALUES (6, 7, '2024-Odd', 35); SAVEPOINT more_internal; --
Enter external marks (now assume data entry error found) INSERT INTO ExamResults (StudentID,
CourseID, Semester, External) VALUES (5, 2, '2024-Odd', 150); -- ERROR: exceeds
CHECK(External <= 60) -- Undo only external marks entry, keep internal marks ROLLBACK TO
SAVEPOINT more_internal; -- Re-enter with correct value UPDATE ExamResults SET External =
48, Grade = 'A' WHERE StudentID = 5 AND CourseID = 2 AND Semester = '2024-Odd'; COMMIT;
```

Practice Exercises

PHASE 7

20 hands-on tasks to reinforce every concept — try them yourself!

Concepts covered: All Concepts • Self-Assessment • Answers at End

Table & Schema Tasks

1 Add a new table

- Create a 'Clubs' table: ClubID, ClubName, FacultyAdvisorID (FK to Professors), EstYear
- Add a 'ClubMemberships' table: StudentID + ClubID (composite PK), JoinedOn
- What relationship type is Student-Club? How do you implement it?

2 Modify existing schema

- Add a 'ProfilePhoto' VARCHAR(300) column to Students table
- Add a 'SemesterOffered' column to Courses (e.g. 'Odd','Even','Both')
- Change DeptCode in Departments from CHAR(4) to CHAR(6) to support longer codes

Row Manipulation Tasks

3 INSERT tasks

- Add 3 new students — one from each of these departments: CSE, MTH, ECE
- Enrol StudentID 8 in CS302 and CS303 for '2024-Odd' semester
- Insert a hostel allotment for the new CSE student in Nilgiri Block, Room N301

4 UPDATE tasks

- A professor changed their email — update Dr. Ravi Menon's email
- Update all students' CGPA after the 2024-Odd semester results are finalized
- Mark all 'Pending' tuition fees older than 60 days as 'Overdue' (add a new status)

5 DELETE tasks

- A course was cancelled — delete CS401 and check what happens to Enrollments (CASCADE?)
- A student withdrew — use soft delete (IsActive=FALSE) instead of hard delete. Why?
- Remove all exam results for a course that was incorrectly entered

Constraint Tasks

6 Test and add constraints

- Try inserting a student with CGPA = 11.5 — what error do you get?
- Try inserting two students with the same RollNo — what happens?
- Add a CHECK constraint to Fees table: Amount must be at least 100
- Add a UNIQUE constraint so no student can live in two hostels (already enforced how?)

Index Tasks

7 Index optimisation

- Run EXPLAIN on: `SELECT * FROM Enrollments WHERE StudentID=1 AND Semester='2024-Odd'`
- Is an index used? If not, create the right composite index and re-run EXPLAIN
- Create an index to speed up: `SELECT * FROM ExamResults WHERE Grade = 'A+'` — is it worth it?

Query Tasks

8 Write these SELECT queries

- List all professors with their department name and total courses they teach
- Find students who are enrolled in 3 or more courses this semester
- Show the hostel occupancy: hostel name, capacity, students allotted, rooms remaining
- Find students who have paid ALL their fees (no pending records)

9 Advanced queries

- Find the course with the highest average exam score
- List students who are in the top 10% CGPA of their department
- Show month-wise fee collection totals for the current year

Transaction Tasks

10 ACID scenarios

- Write a transaction to transfer a student from CSE to ECE dept + update all enrollments
- Write a transaction that fails halfway — verify rollback worked using SELECT
- Simulate two concurrent fee payments: show how isolation prevents double payment
- Use SAVEPOINT to enter exam marks for 5 students — rollback only the 3rd entry if wrong

Self-Assessment Checklist — Can You Do All of These?

PRACTICE CHECKLIST

- ☐ Create a database and switch to it with USE
- ☐ Create a table with all 6 constraint types: PK, FK, NOT NULL, UNIQUE, CHECK, DEFAULT
- ☐ Create a 1:N relationship between two tables using a Foreign Key
- ☐ Create an M:N relationship using a junction table with composite PK
- ☐ Create a 1:1 relationship using UNIQUE FK
- ☐ Insert a row that deliberately violates each constraint and observe the error
- ☐ Write an INSERT ... SELECT to copy data between tables
- ☐ Write an UPDATE that affects multiple rows with a WHERE condition
- ☐ Add an index and verify it is used with EXPLAIN
- ☐ Write a 3-table JOIN query returning data from all three tables
- ☐ Write a GROUP BY query with HAVING to filter groups
- ☐ Write a subquery in a WHERE clause
- ☐ Wrap a multi-step operation in BEGIN / COMMIT
- ☐ Trigger a ROLLBACK and verify the database state is unchanged
- ☐ Use SAVEPOINT and ROLLBACK TO to partially undo a transaction

Quick Reference — All Tables in the Project

Table	Primary Key	Foreign Keys	Key Constraints	Relationship
Departments	DeptID (AUTO)	—	DeptName UNIQUE, DeptCode UNIQUE	Parent table
Hostels	HostelID (AUTO)	—	HostelName UNIQUE, Capacity > 0	Parent table
Professors	ProfID (AUTO)	DeptID → Departments	Email UNIQUE, Salary >= 25000	1:M with Departments
Students	StudentID (AUTO)	DeptID → Departments	RollNo UNIQUE, Email UNIQUE	1:M with Departments
Courses	CourseID (AUTO)	ProfID → Professors, DeptID → Departments	CourseCode UNIQUE, Credits 1-6	1:M with Professors
Enrollments	StudentID+CourseID+Semester	StudentID → Students, CourseID → Courses, Semester → Sections	SeatID → ENUM, CASCADE delete	M:N Students-Courses
ExamResults	ResultID (AUTO)	StudentID → Students, CourseID → Courses, Semester → Sections	Score 0-100, External 0-60	M:N extended
HostelAllotments	AllotID (AUTO)	StudentID → Students, HostelID → Hostels	StudentID UNIQUE (1:1)	1:1 Students-Hostels
Fees	FeeID (AUTO)	StudentID → Students	Amount > 0, Status ENUM	N:1 with Students

Concept → Where You Learned It In This Project

Concept	Where in Project	What You Did
Table	Phase 1 + Phase 2	Identified 9 entities; created each as a table
Row	Phase 3	Inserted 8 students, 5 professors, results, fees
Column	Phase 2	Defined columns with names, types, and constraints in every table
Primary Key	Phase 2 Step 2-3	AUTO_INCREMENT PK on all tables; composite PK on Enrollments
Foreign Key	Phase 2 Step 4	Linked Professors → Depts, Students → Depts, Enrollments → both
Schema	Phase 1 + Phase 2	Designed 9-table schema; drew relationship map
Index	Phase 4	Created 7 indexes; tested with EXPLAIN
Constraints	Phase 2 + Phase 3 Step 9	Used all 6 types; tested violations
Relationships	Phase 2 Steps 3-4	Built 1:1 (hostel), 1:N (dept-students), M:N (enrollments)
ACID	Phase 6	3 transactions: enrolment, fee payment, exam results with SAVEPOINT

Extend the Project — Next Challenges

Extension	Tables to Add	New Concepts
Library System	Books, BookIssues	M:N Students-Books; fine calculation
Timetable	ClassRooms, TimeSlots, Schedules	3-way composite FK; conflict detection
Online Exams	Questions, StudentResponses, AutoGrades	15 ON columns; triggers for auto-grading
Notifications	NotifLog, NotifTemplates	Stored procedures; event scheduler
Reports Dashboard	(Views + Aggregates)	CREATE VIEW; indexed views; CTEs